Introdução ao Streamlit

1 Preparando o ambiente

Crie a pasta cadastro_fornecedores e um arquivo app.py. No terminal, instale as bibliotecas:

```
python -m pip install streamlit pandas
```

Experimente: confirme que a instalação foi concluída no ambiente Python que será utilizado na atividade.
Adicione ao app.py uma importação e um título, conforme o próximo exemplo. Para iniciar a aplicação, execute na pasta do projeto:

```
python -m streamlit run app.py
```

Experimente: abra o endereço indicado no terminal, normalmente http://localhost:8501. Após preencher o arquivo com o exemplo seguinte, salve e observe a interface.
Utilize o mesmo app.py durante a atividade. Nos exemplos iniciais, acrescente os trechos ao arquivo. Quando chegar ao formulário, substitua os campos de teste anteriores para evitar componentes duplicados.

2 Apresentando informações na interface

Os componentes textuais estabelecem diferentes níveis de destaque. Comece com títulos, subtítulos e uma breve apresentação:

```
import streamlit as st

st.title("Sistema de Gestão de Fornecedores")
st.header("Cadastro de Empresas")
st.subheader("Informações Cadastrais")
st.write("Aplicação desenvolvida utilizando Python e Streamlit.")
```

Experimente: altere a ordem dos componentes. Acrescente um segundo st.subheader() e utilize st.write() para exibir uma informação numérica.
Markdown permite formatar textos; legendas, texto simples e blocos de código atendem a outras necessidades de apresentação.

```
st.markdown("**Cadastro de Fornecedores**")
st.markdown("*Gerenciamento de informações empresariais*")
st.caption("Sistema desenvolvido para fins acadêmicos.")
st.text("Versão 1.0")
st.code("print('Olá, Streamlit!')", language="python")
```

Experimente: adicione uma lista com três funcionalidades do sistema utilizando Markdown e inclua um link para a documentação oficial do Streamlit.

3 Mensagens e indicadores visuais

Mensagens informam o resultado de operações e sinalizam situações que exigem atenção.

```
st.info("Preencha os dados para realizar um cadastro.")
st.success("Operação realizada com sucesso!")
st.warning("Algumas informações ainda não foram preenchidas.")
st.error("Não foi possível concluir a operação.")
```

Experimente: observe as diferenças visuais e mantenha apenas a mensagem mais adequada para a apresentação inicial do cadastro.
Indicadores apresentam valores resumidos. Os números abaixo são ilustrativos.

```
st.metric(
    label="Fornecedores Cadastrados",
    value=25,
    delta=5
)
```

Experimente: adicione outro indicador para fornecedores ativos. Altere os valores e observe a apresentação da variação.

4 Organizando a interface

Containers agrupam elementos relacionados e podem receber uma borda.

```
with st.container(border=True):
    st.subheader("Informações do Sistema")
    st.write("Módulo de Cadastro de Fornecedores")
    st.write("Armazenamento em arquivo CSV")
```

Experimente: crie um segundo container com o título Informações Complementares. Compare border=True e border=False.
Colunas distribuem os elementos horizontalmente.

```
col1, col2 = st.columns(2)

with col1:
    st.subheader("Dados Cadastrais")
    st.write("Razão Social")
    st.write("CNPJ")

with col2:
    st.subheader("Dados de Contato")
    st.write("Telefone")
    st.write("E-mail")
```

Experimente: utilize três colunas e distribua informações de cadastro, contato e endereço entre elas.
Uma lista de proporções define larguras diferentes para as colunas.

```
col1, col2 = st.columns([3, 1])
with col1:
    st.write("Nome da Empresa")
with col2:
    st.write("Código")
```

Experimente: compare as proporções [2, 1] e [1, 2], observando a distribuição do espaço.

Divisores separam conteúdos; áreas expansíveis recolhem informações complementares.

```
st.divider()
with st.expander("Sobre o sistema"):
    st.write(
        "Esta aplicação permite cadastrar fornecedores "
        "e armazenar suas informações em arquivo CSV."
    )
```

Experimente: adicione uma área expansível contendo orientações para o preenchimento dos dados cadastrais.

5 Recebendo informações do usuário

Campos textuais retornam os valores informados. Observe quando a interface é atualizada ao editar e confirmar uma entrada.

```
razao_social = st.text_input("Razão Social")
nome_fantasia = st.text_input("Nome Fantasia")
st.write("Razão Social informada:", razao_social)
st.write("Nome Fantasia informado:", nome_fantasia)
```

Experimente: acrescente campos para CNPJ, telefone e e-mail. Apresente os valores com st.write().
Seletores oferecem opções predefinidas; caixas de marcação retornam valores booleanos.

```
estado = st.selectbox("Estado", ["GO", "DF", "MG", "SP", "RJ"])
isento = st.checkbox("Isento de Inscrição Estadual")
st.write("Estado selecionado:", estado)
st.write("Isento:", isento)
```

Experimente: amplie a lista de estados e observe o retorno ao marcar e desmarcar a opção de isenção.
Botões permitem condicionar uma operação à interação do usuário.

```
nome = st.text_input("Nome do Fornecedor")
if st.button("Confirmar"):
    st.success(f"Fornecedor {nome} confirmado!")
```

Experimente: mostre a confirmação apenas quando o nome estiver preenchido. Caso contrário, apresente uma mensagem de erro.

6 Construindo o formulário de cadastro

Substitua os exemplos anteriores de interface pelo código abaixo. O formulário reúne os campos e envia seus valores em conjunto quando Salvar é acionado.

```
import streamlit as st

st.title("Cadastro de Fornecedores")
with st.form("form_fornecedor"):
    st.subheader("Informações Principais")
    razao_social = st.text_input("Razão Social *")
    nome_fantasia = st.text_input("Nome Fantasia")
    col1, col2 = st.columns(2)
    with col1:
        cnpj = st.text_input("CNPJ")
    with col2:
        inscricao_estadual = st.text_input("Inscrição Estadual")
    isento = st.checkbox("Isento")
    salvar = st.form_submit_button("Salvar")

if salvar:
    st.write("Fornecedor:", razao_social)
    st.write("Nome Fantasia:", nome_fantasia)
    st.write("CNPJ:", cnpj)
    st.write("Inscrição Estadual:", inscricao_estadual)
    st.write("Isento:", isento)
```

Experimente: complete o formulário com os campos a seguir. Organize as seções utilizando containers, colunas e divisores; compare o comportamento dos campos dentro e fora do formulário.

- Informações principais: Razão Social, Nome Fantasia, CNPJ, Inscrição Estadual e Isento.
- Informações complementares: Telefone, Fax, Contato e E-mail.
- Endereço da empresa: Logradouro, Número, Bairro, Complemento, Cidade, Estado e CEP.

Procure uma distribuição visual semelhante à referência da atividade, com os três grupos de informações claramente identificados.

7 Validando os dados do formulário

Substitua o bloco if salvar anterior pelo trecho abaixo. O asterisco no rótulo, por si só, não torna um campo obrigatório; a regra é implementada no código.

```
if salvar:
    if not razao_social.strip():
        st.error("Informe a Razão Social.")
    elif not cnpj.strip():
        st.error("Informe o CNPJ.")
    else:
        st.success("Dados validados com sucesso!")
```

Experimente: teste campos vazios. Acrescente uma verificação de 14 dígitos para o CNPJ usado nos exemplos, removendo a pontuação. Essa verificação de formato não valida os dígitos verificadores.

8 Armazenando informações em arquivo CSV

CSV representa dados tabulares em texto. Utilize a biblioteca csv do Python para

adicionar registros sem sobrescrever os anteriores. Coloque as importações e a função antes do formulário e de sua chamada.

```
import csv
from pathlib import Path

arquivo_csv = Path("fornecedores.csv")

def salvar_fornecedor(dados):
    criar_cabecalho = (
        not arquivo_csv.exists()
        or arquivo_csv.stat().st_size == 0
    )
    with arquivo_csv.open(
        mode="a", newline="", encoding="utf-8"
    ) as arquivo:
        escritor = csv.DictWriter(arquivo, fieldnames=dados.keys())
        if criar_cabecalho:
            escritor.writeheader()
        escritor.writerow(dados)
```

Experimente: teste a função com um dicionário de três informações fictícias. Abra o CSV e repita com outro fornecedor. Retire a chamada de teste do app.py depois do experimento, pois a reexecução do script repetiria a gravação.
Antes de integrar o formulário completo, exclua somente o CSV de teste ou utilize outro nome de arquivo. O CSV definitivo deve começar com o conjunto completo de colunas, pois adicionar campos ao dicionário não atualiza um cabeçalho já gravado.
Substitua o tratamento do botão Salvar pelo código abaixo e mantenha a função de persistência definida anteriormente.

```
if salvar:
    if not razao_social.strip():
        st.error("Informe a Razão Social.")
    elif not cnpj.strip():
        st.error("Informe o CNPJ.")
    else:
        fornecedor = {
            "razao_social": razao_social.strip(),
            "nome_fantasia": nome_fantasia.strip(),
            "cnpj": cnpj.strip(),
            "inscricao_estadual": inscricao_estadual.strip(),
            "isento": isento
        }
        salvar_fornecedor(fornecedor)
        st.success("Fornecedor cadastrado com sucesso!")
```

Experimente: amplie o dicionário com todos os campos do formulário e preserve a validação desenvolvida anteriormente. Cadastre pelo menos cinco fornecedores fictícios e confira o arquivo.
Mantenha os mesmos nomes de colunas e a mesma ordem em todas as gravações. Quando Isento estiver marcado, defina uma regra coerente para armazenar a inscrição estadual.
Execute a atividade localmente. O arquivo será gravado no ambiente do servidor

9 Consultando os fornecedores cadastrados

Adicione a consulta após o tratamento do formulário. A leitura como texto preserva zeros à esquerda de documentos e códigos postais.

```
import pandas as pd

st.subheader("Fornecedores Cadastrados")
if arquivo_csv.exists() and arquivo_csv.stat().st_size > 0:
    df = pd.read_csv(
        arquivo_csv, dtype=str, keep_default_na=False
    )
    st.dataframe(df, use_container_width=True)
else:
    st.info("Nenhum fornecedor cadastrado.")
```

Experimente: cadastre novos fornecedores e observe a tabela. Apresente também a quantidade total de registros, tratando o caso em que o arquivo ainda não existe.

10 Finalização

Ao concluir a atividade, a aplicação deverá apresentar as três seções do cadastro, validar os campos obrigatórios, salvar os registros em CSV e permitir a consulta dos fornecedores cadastrados.
Como desafio final, impeça o cadastro de CNPJ repetido, comparando apenas os dígitos dos documentos utilizados nos exemplos. Teste também o comportamento quando o CSV ainda não existe.